claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_095035a0-e9f\agent-a9cb486a469b7e268.jsonl`
- SHA-256: `6b58b8b3deb9ac69c219d0ea85764ddfe5c5e3c4b51fc06ac78a2472f6b5b67d`
- Source modified: `2026-06-13T19:18:56+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf_095035a0-e9f`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-13T19:13:53.212Z)

Adversarially VERIFY this fusion-P2 review finding by reading the actual code. Try to REFUTE it; only confirm real=true if it genuinely holds against the code. Default to real=false if uncertain or if it's a nitpick/false-positive.

FINDING: {"title":"Docstring overstates the torch-free guarantee: importing the segmenter registry DOES pull torch (via yolo_hybrid, pre-existing)","severity":"low","file":"backend/pipeline/segmenters/fusion_hybrid.py:12-13, 91 (and __init__.py + yolo_hybrid.py:25)","detail":"Verified empirically: in a fresh interpreter, `import backend.pipeline.segmenters` (and even `import backend.pipeline.segmenters.fusion_hybrid`) leaves torch in sys.modules. Cause: segmenters/__init__.py imports yolo_hybrid, and yolo_hybrid.py:25 does a TOP-LEVEL `from backend.pipeline.detectors.person_detector import PersonDetector`, and person_detector.py:27 does `import torch` at module top. fusion_hybrid's OWN four top-level imports (wasb_runner, tracknet_runner, rally_gate, fusion_features) are genuinely torch-free (traced in isolation: each shows torch=False), and its PersonDetector import is correctly lazy inside _person_features (fusion_hybrid.py:92). So the fusion change neither adds nor removes torch from the registry path. But the docstring framing ('the DEFAULT path never imports torch / touches the GPU', repeated at lines 12-13 and 91) is misleading at the registry level ? torch is already eagerly loaded by yolo_hybrid before fusion is even reached. Note this is a pre-existing condition in yolo_hybrid, not a bug introduced by this PR; flagging only because the prompt explicitly asked whether importing the registry pulls torch (answer: yes, via yolo_hybrid).","fix":"No change required in fusion_hybrid. If the torch-free-default-path guarantee is meant to be real for the whole registry, make yolo_hybrid.py import PersonDetector lazily too (move the import into __init__/use sites), as person_detector itself was designed for (its module docstring claims heavy deps are lazy, but `import torch` at line 27 is eager). Otherwise soften the fusion docstring to scope the claim to 'this module' rather than 'the default path'."}

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/person_detector.py, backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:16:18.743Z)

I'll verify this finding by reading the actual code and tracing the import chain.

3. user (2026-06-13T19:16:22.717Z)

```
1       """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3       Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4       TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5       do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6       shuttle-seeded window via two overridable seams the engine exposes:
7
8       - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9         GPU-free, computed from the trajectory we already have); and, only when the person cue
10        is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11        (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
12        fusion (P3). The person path lazily builds ONE `PersonDetector` ? so the default path
13        never imports torch / touches the GPU.
14      - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
15        (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
16        dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
17        **Persisted candidates remain the full superset** regardless, so the offline
experiment
18        harness can re-score any variant.
19
20      Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
21      nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
22      and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
23      segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
24      experiment harness (EXPERIMENT_HARNESS.md), never silently.
25      """
26      from typing import Any, Dict, List, Optional, Tuple
27
28      from backend.pipeline.segmenters.base import segmenter_registry
29      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
30      from backend.pipeline.detectors.base import DetectorRunner
31      from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
32      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
33      from backend.pipeline.detectors import rally_gate
34      from backend.pipeline.detectors import fusion_features as ff
35
36
37      class FusionSegmenter(TrajectoryHybridSegmenter):
38          """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
39
40          family = "fusion"
41          display_label = "Fusion"
42
43          def __init__(self, db: Any, config: Any):
44              super().__init__(db, config)
45              # Built lazily only if the person cue is enabled (keeps torch/GPU off the
default path).
46              self._person: Optional[Any] = None
47
48          @property
49          def name(self) -> str:
50              return "fusion_hybrid"
51
52          @property
53          def description(self) -> str:
54              return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
```

```
rallies, "
55                      "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
56                      "AI provider sets semantic boundaries.")
57
58          def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
59              substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
60              if substrate == "tracknet":
61                  cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
62                  return TrackNetRunner(cfg), {
63                      "weights_path": cfg.weights_path,
64                      "queue_length": cfg.queue_length,
65                      "fusion_substrate": "tracknet",
66                  }
67              wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
68              return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
69
70          def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
71                                      frame_width: float, video_path: str,
72                                      idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
73              stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
74              # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
75              # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
76              # over the whole trajectory by rally_gate (camera-agnostic).
77              gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
min_crossings=0)
78              stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
79
80              fcfg = idx_cfg.get("fusion", {})
81              if fcfg.get("cue_flags", {}).get("person", False):
82                  stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
83              return stats
84
85          def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
86                               frame_width: float, video_path: str,
87                               fcfg: Dict[str, Any]) -> Dict[str, float]:
88              """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
89              frame range and compute the scale/translation-invariant person features. Lazily
builds
90              ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
91              # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
92              from backend.pipeline.detectors.person_detector import PersonDetector
93              if self._person is None:
94                  self._person = PersonDetector(self.config)
95
96              frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
97              start_f, end_f = int(cw["start"] * fps), int(cw["end"] * fps)
98              det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
99              fw = det["frame_width"] or frame_width
100             fh = det["frame_height"] or 0.0
```

4. user (2026-06-13T19:16:26.058Z)

```
1       from backend.pipeline.segmenters.base import segmenter_registry, BasePlaySegmenter
2       from backend.pipeline.segmenters.motion import MotionPlaySegmenter
3       from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
```

```
from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter

__all__ = [
    "segmenter_registry",
    "BasePlaySegmenter",
    "MotionPlaySegmenter",
    "GeminiPlaySegmenter",
    "HybridYoloSegmenter",
    "TrackNetHybridSegmenter",
    "WasbHybridSegmenter",
    "FusionSegmenter",
]

```

5. user (2026-06-13T19:16:29.248Z)

```
"""Two-stage rally segmenter: torchvision person detection + ByteTrack tracking
(Stage 1, local) feeding an AI provider for semantic boundaries (Stage 2).

Stage-1 detection now lives in `backend/pipeline/detectors/person_detector.py`
(`PersonDetector`, a reusable rally-cue producer ? fusion P1) so the SAME person cue
can be consumed by the future fusion segmenter; this segmenter just orchestrates
chunking, the AI-provider handoff, and DB population around it. Behaviour is unchanged.

Stage 1 originally used the AGPL-licensed ultralytics YOLO. To keep the hosted
service free of copyleft obligations (see docs/MONETIZATION_AUDIT.md and ADR-005),
detection uses torchvision's permissively-licensed (BSD) detectors and the
association/tracking that YOLO.track() used to bundle in is provided by
supervision's MIT-licensed ByteTrack.

The registry key stays "yolo_hybrid" for back-compat with existing configs and the
DB `detector` tags, even though no YOLO is involved anymore.
"""
import os
import time
import logging
import concurrent.futures
from typing import List, Dict, Any, Tuple

from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
from backend.pipeline.detectors.person_detector import PersonDetector, _DEFAULT_DETECTOR
from backend.utils.video import get_video_duration, extract_video_chunk
from backend.sports import sport_registry
from backend.providers import provider_registry

logger = logging.getLogger(__name__)


def _fmt_ts(seconds: float) -> str:
    """Format seconds as HH:MM:SS for cross-verifying a timestamp against the video."""
    seconds = max(0, int(seconds))
    h, rem = divmod(seconds, 3600)
    m, s = divmod(rem, 60)
    return f"{h:02d}:{m:02d}:{s:02d}"


```

6. user (2026-06-13T19:16:33.291Z)

```
"""Person-detection rally cue ? torchvision detector + ByteTrack + motion windowing.

Extracted verbatim from `yolo_hybrid` (fusion P1, `docs/MULTI_SIGNAL_FUSION_PLAN.md`):
```

```
4       the **person cue is a swappable producer**, not logic baked into one segmenter. Today
5       `HybridYoloSegmenter` is the only consumer (it delegates Stage-1 here, byte-
identically);
6       next the fusion segmenter consumes the SAME producer over shuttle-seeded windows, and
7       later the learned classifier reads its persisted features. Keeping detection here (not
in
8       the segmenter) is what makes "add a cue = register a producer" true.
9
10      Behaviour is unchanged from the in-lined version ? only the home moved:
11        - `lazy_load_model`               ? load a permissively-licensed torchvision detector
12        - `make_tracker`                  ? a fresh supervision ByteTrack per chunk
13        - `detect_and_track`              ? one frame ? confident persons with persistent IDs
14        - `extract_action_windows_from_chunk` ? a chunk ? high-motion candidate windows
15          with the motion stats (`peak/mean_velocity`, `active_frame_count`, `track_id_count`)
16          that feed the candidate `stats` JSON and the learned fusion features.
17
18      Heavy deps (torchvision, supervision) are imported lazily inside the methods so
importing
19      the segmenter registry never pulls torch on a machine that lacks it (and tests can mock
20      the seams). Licensing: torchvision detectors are BSD + COCO weights; ByteTrack is MIT
21      (ADR-005; ultralytics' AGPL YOLO was dropped).
22      """
23      import logging
24      from typing import Any, Dict, List, Optional, Tuple
25
26      import cv2
27      import torch
28
29      from backend.utils.geometry import get_velocity_normalised
30
31      logger = logging.getLogger(__name__)
32
33      # torchvision detection models are trained on COCO where the "person" category is
34      # class 1 (index 0 is the implicit background). NB: ultralytics YOLO used class 0 ?
35      # this off-by-one is the classic gotcha when swapping detectors. Keep it named.
36      _PERSON_CLASS_ID = 1
37
38      # Permissively-licensed torchvision detectors selectable via config
39      # `indexing.detector_model`. All are BSD-licensed (code + COCO weights).
40      _DETECTOR_FACTORIES = {
```

7. assistant (2026-06-13T19:16:39.763Z)

Let me verify the empirical claim about the import chain by actually running it, and check the rest of fusion_hybrid plus the diffs.

8. user (2026-06-13T19:16:41.312Z)

```
1       """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3       Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4         1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5         2. copies wasb_infer.py into the WASB repo's src/ dir,
6         3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7         4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9       So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10      with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11      """
12      import os
13      import shlex
14      import logging
15      import subprocess
16      from dataclasses import dataclass
17      from typing import Dict, Any, Optional, Tuple
```

```
18
19      # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20      # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
TrackNetRunner.
21      from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
22      from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
23
24      logger = logging.getLogger(__name__)
25
26      # Windows path to the inference wrapper we stage into WSL.
27      _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
28
29
30      @dataclass
```

9. user (2026-06-13T19:16:42.197Z)

```
1       """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3       This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4       inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6           1. Translate the Windows video path to its WSL-visible form (and optionally
7              stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8           2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9           3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10          4. Convert that trajectory into high-motion "action windows" ? the same
11             candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12             pipeline (AI handoff, DB population) is unchanged.
13
14      Nothing here imports TensorFlow; all model code stays in WSL.
15      """
16
17      import os
18      import csv
19      import shlex
20      import logging
21      import subprocess
22      from dataclasses import dataclass
23      from typing import List, Dict, Any, Optional, Tuple
24
25      from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27      logger = logging.getLogger(__name__)
28
29
30      @dataclass
```

10. user (2026-06-13T19:16:43.212Z)

```
1       """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3       WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4       the opponent for service after losing a point, that single trajectory looks like a
5       rally. The discriminator is structural, not duration-based: a real rally has the
6       shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7       single service feed crosses ~once.
8
9       This module is camera-agnostic and needs NO new model ? it works on the trajectory
10      we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11      axis with the larger shuttle spread) and the net position (median shuttle coord on
12      that axis ? the shuttle clusters at the two ends and crosses the net region often,
13      so the median sits near the net), then counts sign changes of (coord - net) across
```

```
14      the visible points inside a candidate window, with a deadband to ignore jitter.
15
16      Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17      Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18      windows from trajectory_to_action_windows, or to fold into it later.
19      """
20      from __future__ import annotations
21
22      from dataclasses import dataclass
23      from typing import List, Sequence, Tuple
24
25      Interval = Tuple[float, float]
26
27
28      @dataclass
29      class GatedWindow:
30          start: float
```

11. user (2026-06-13T19:16:43.582Z)

```
1       """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3       NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4       math is unit-testable in a GPU-free dev container. The detection (person boxes,
5       shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6       module only turns those signals into a handful of **scale/translation-invariant**
7       features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8       memorise this court/camera and fail to generalise off a small labelled set).
9
10      Conventions:
11      - A person box is ``(x1, y1, x2, y2)`` in **pixels**; ``(track_id, box)`` when motion
needs it.
12      - ``per_frame`` is ``{frame_idx: [(track_id, box), ...]}`` keyed by the ORIGINAL video
frame
13        index (so it aligns directly with the shuttle trajectory's ``.frame`` ? no clip re-
timing).
14      - A shuttle point has ``.frame``, ``.visible``, ``.x``, ``.y`` (pixels) ? the WASB
trajectory shape.
15      - ``court_polygon`` is normalised 0-1 ``[(x, y), ...]`` or ``None`` (None ? no mask,
every
16        detection counts ? the player-count-only mode).
17      - ``net`` is ``(axis 'x'|'y', position 0-1 normalised)`` or ``None`` (None ? two-sided =
0.0).
18
19      Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20      floats in [0, 1] except ``mean_player_motion`` (a non-negative normalised speed).
21      """
22      from __future__ import annotations
23
24      from typing import Dict, List, Optional, Sequence, Tuple
25
26      BBox = Tuple[float, float, float, float]
27      Net = Tuple[str, float]
28
29
30      def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
```

12. assistant (2026-06-13T19:16:47.212Z)

The code reading confirms the structural claims. Now let me empirically verify the core assertion: does importing the segmenter registry pull torch into sys.modules. The dev container lacks torch, so I'll trace the import statements rather than rely on actual torch presence ? but let me check whether torch is even importable, and trace the chain precisely.

13. user (2026-06-13T19:16:48.898Z)

```
torch installed: True
```

14. assistant (2026-06-13T19:16:51.426Z)

torch is actually installed here. Let me run the empirical test the finding claims.

15. user (2026-06-13T19:17:00.019Z)

```
torch pre-import: False
torch after import backend.pipeline.segmenters: True
```

16. user (2026-06-13T19:17:06.316Z)

```
torch pre-import: False
torch after import fusion_hybrid: True
```

17. assistant (2026-06-13T19:17:11.093Z)

Both empirical claims confirmed. Now let me verify the nuanced part: the finding claims fusion_hybrid's OWN four top-level imports are torch-free, and that torch only comes in via the `__init__.py` importing yolo_hybrid (when importing the package). But the second test shows `import backend.pipeline.segmenters.fusion_hybrid` ALSO pulls torch. Let me determine WHY ? the finding says fusion_hybrid's own imports are torch-free, so importing the submodule directly should still trigger `__init__.py` (Python runs the package `__init__` first). Let me verify the precise causal chain.

18. user (2026-06-13T19:17:15.158Z)

(Bash completed with no output)

19. assistant (2026-06-13T19:17:18.289Z)

Let me run each import in a fresh interpreter to isolate whether each module pulls torch on its own. The key question is whether `trajectory_hybrid` (the base class fusion subclasses) pulls torch, since that's also a top-level import of fusion_hybrid.

20. user (2026-06-13T19:17:34.893Z)

```
backend.pipeline.detectors.wasb_runner -> torch: False
backend.pipeline.detectors.tracknet_runner -> torch: False
backend.pipeline.detectors.rally_gate -> torch: False
backend.pipeline.detectors.fusion_features -> torch: False
backend.pipeline.segmenters.trajectory_hybrid -> torch: True
backend.pipeline.segmenters.base -> torch: True
```

21. assistant (2026-06-13T19:17:39.025Z)

Interesting ? this partially contradicts the finding's claim. The finding says fusion_hybrid's "OWN four top-level imports (wasb_runner, tracknet_runner, rally_gate, fusion_features) are genuinely torch-free." That's true for those four. But fusion_hybrid also has a FIFTH top-level import: `trajectory_hybrid` (line 29), and that pulls torch=True. And `base` also pulls torch=True. Let me trace why `trajectory_hybrid` and `base` pull torch, since these are the segmenter base classes fusion_hybrid directly imports.

22. user (2026-06-13T19:17:39.906Z)

```
1       """Shared engine for trajectory-substrate hybrid segmenters.
2
3       tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4       standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5       now the single home for the whole pipeline shape:
```

```
6
7           healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8           -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10      A concrete subclass supplies only its identity and runner wiring:
11      - ``family``        ? config section under ``indexing.*``, output subdir, and
12                            the ``<family>-hybrid-<model>`` detector tag in metadata.
13      - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14      - ``name`` / ``description`` properties (the registry contract).
15      - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16                            detection_params extras such as weights_path).
17
18      This base is deliberately NOT registered; only concrete subclasses register.
19      """
20      import os
21      import time
22      import logging
23      import concurrent.futures
24      from typing import Any, Dict, List, Optional, Tuple
25
26      import cv2
27
28      from backend.pipeline.segmenters.base import BasePlaySegmenter
29      from backend.utils.video import get_video_duration, extract_video_chunk
30      from backend.sports import sport_registry
31      from backend.providers import provider_registry
32      from backend.pipeline.detectors.base import DetectorRunner
33
34      logger = logging.getLogger(__name__)
35
36
37      def _fmt_ts(seconds: float) -> str:
38          seconds = max(0, int(seconds))
39          h, rem = divmod(seconds, 3600)
40          m, s = divmod(rem, 60)
```

23. user (2026-06-13T19:17:40.630Z)

```
1       from abc import ABC, abstractmethod
2       from typing import List, Dict, Any, Type, Optional
3       from backend.infrastructure.database import Database
4       from backend.results import Result  # new standardized contract (Result[Value] ==
Success[Value] | Failure)
5
6       class BasePlaySegmenter(ABC):
7           def __init__(self, db: Database, config: Dict[str, Any]):
8               self.db = db
9               self.config = config
10
11          @property
12          @abstractmethod
13          def name(self) -> str:
14              """Unique identifier for this segmenter."""
15              pass
16
17          @property
18          @abstractmethod
19          def description(self) -> str:
20              """Human-readable explanation of this segmenter."""
21              pass
22
23          @abstractmethod
24          def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25              """
```

```
26              Processes a video to detect play segments, populates SQLite database,
27              and returns a Result.
28
29              Use isinstance(result, Failure) (or result.is_success()) to distinguish
30              "no segments found" from "something went wrong" (missing key, healthcheck
31              failure, etc.). This is the start of the standardized error/result contract.
32              """
33              pass
34
35      class SegmenterRegistry:
36          def __init__(self):
37              self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39          def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40              """Registers a new segmenter class under its name property."""
```

24. assistant (2026-06-13T19:17:44.750Z)

`base.py` imports `backend.results` and `backend.infrastructure.database`. One of those pulls torch. `trajectory_hybrid` imports `base` plus `backend.sports`, `backend.providers`, `backend.utils.video`. Let me find the precise source of torch in the base/trajectory chain, because this is material: it means even fusion_hybrid's OWN direct imports (not just the `__init__.py` ? yolo_hybrid path) pull torch. This would mean the finding's causal attribution is partly wrong.

25. user (2026-06-13T19:17:49.181Z)

```
backend.results -> torch: False
backend.infrastructure.database -> torch: False
backend.sports -> torch: False
backend.providers -> torch: False
backend.utils.video -> torch: False
```

26. assistant (2026-06-13T19:17:53.591Z)

None of these pull torch on their own. So `base.py` and `trajectory_hybrid.py` showing torch=True in isolation must be from something else. Wait ? when I imported `backend.pipeline.segmenters.trajectory_hybrid` and `backend.pipeline.segmenters.base`, Python first executes the package `__init__.py` for `backend.pipeline.segmenters`, which imports yolo_hybrid ? person_detector ? torch. That's the confound. Let me verify by importing the leaf module file in a way that doesn't trigger the package `__init__`, or better, check whether the parent package `__init__` is the culprit.

27. user (2026-06-13T19:18:03.060Z)

```
after import segmenters.base, yolo_hybrid in sys.modules: True
after import segmenters.base, person_detector in sys.modules: True
torch: True
```

28. assistant (2026-06-13T19:18:07.598Z)

Confirmed. The `trajectory_hybrid -> torch: True` and `base -> torch: True` results were NOT because those modules import torch ? they were because importing any submodule of `backend.pipeline.segmenters` first runs the package `__init__.py`, which imports `yolo_hybrid` ? `person_detector` ? `import torch`. So the finding's causal attribution is correct: torch comes in via `__init__.py` ? `yolo_hybrid` ? `person_detector:27`, NOT via fusion_hybrid's own logic.

Now let me verify the exact line numbers the finding cites and confirm fusion_features and person_detector edits. Let me check person_detector line 27 and the diff to confirm `import torch` is eager and pre-existing.

29. user (2026-06-13T19:18:09.241Z)

```
diff --git a/backend/pipeline/detectors/person_detector.py
b/backend/pipeline/detectors/person_detector.py
index 38ec2b3..5b931a4 100644
--- a/backend/pipeline/detectors/person_detector.py
+++ b/backend/pipeline/detectors/person_detector.py
@@ -252,3 +252,50 @@ class PersonDetector:
         windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]

         return windows
+
+    def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
+                             frame_skip: int = 3) -> Dict[str, Any]:
+        """Per-frame person detections over ``[start_frame, end_frame]`` of the ORIGINAL video.
+
+        Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
+        ``extract_action_windows_from_chunk`` (which detects ITS OWN windows and discards the
+        per-frame boxes), this returns the raw per-frame detections so the fusion features can
+        align them with the shuttle trajectory. It reads the source by **absolute frame index**
+        (seek + sequential read), NOT a re-extracted clip, so the returned frame indices match
+        the WASB trajectory's ``.frame`` exactly ? that alignment is what
+        ``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.
+
+        Returns ``{"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
+        "frame_height"}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-seek
+        accuracy varies by codec (it lands near the requested keyframe then decodes forward);
+        treat the indices as ±a few frames on real footage ? fine for window-level features.
+        """
+        self.lazy_load_model()
+        cap = cv2.VideoCapture(video_path)
+        if not cap.isOpened():
+            logger.error(f"Could not open {video_path}")
+            return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
+
+        fps = cap.get(cv2.CAP_PROP_FPS)
+        if fps <= 0:
+            fps = 30.0
+        frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
+        frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
+        score_thresh = self.config.get("indexing", {}).get("detector_score_threshold", 0.5)
+        step = max(frame_skip, 1)
+        tracker = self.make_tracker(fps / step)
+
+        start_frame = max(0, int(start_frame))
+        frames: Dict[int, List[Tuple[int, Tuple[float, float, float, float]]]] = {}
+        cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
+        frame_idx = start_frame
+        while frame_idx <= end_frame:
+            ret, frame = cap.read()
+            if not ret:
+                break
+            if (frame_idx - start_frame) % step == 0:
+                frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
+            frame_idx += 1
+        cap.release()
+        return {"frames": frames, "fps": fps,
+                "frame_width": float(frame_width), "frame_height": float(frame_height)}
```

30. user (2026-06-13T19:18:10.554Z)

```
df7d5df refactor(detectors): extract the person cue into PersonDetector (fusion P1) (#115)
df7d5df refactor(detectors): extract the person cue into PersonDetector (fusion P1) (#115)
diff --git a/backend/pipeline/detectors/person_detector.py
b/backend/pipeline/detectors/person_detector.py
new file mode 100644
index 0000000..38ec2b3
```

```
--- /dev/null
+++ b/backend/pipeline/detectors/person_detector.py
@@ -0,0 +1,254 @@
+"""Person-detection rally cue ? torchvision detector + ByteTrack + motion windowing.
+
+Extracted verbatim from `yolo_hybrid` (fusion P1, `docs/MULTI_SIGNAL_FUSION_PLAN.md`):
+the **person cue is a swappable producer**, not logic baked into one segmenter. Today
+`HybridYoloSegmenter` is the only consumer (it delegates Stage-1 here, byte-identically);
+next the fusion segmenter consumes the SAME producer over shuttle-seeded windows, and
+later the learned classifier reads its persisted features. Keeping detection here (not in
+the segmenter) is what makes "add a cue = register a producer" true.
+
+Behaviour is unchanged from the in-lined version ? only the home moved:
+  - `lazy_load_model`               ? load a permissively-licensed torchvision detector
+  - `make_tracker`                  ? a fresh supervision ByteTrack per chunk
+  - `detect_and_track`              ? one frame ? confident persons with persistent IDs
+  - `extract_action_windows_from_chunk` ? a chunk ? high-motion candidate windows
+    with the motion stats (`peak/mean_velocity`, `active_frame_count`, `track_id_count`)
+    that feed the candidate `stats` JSON and the learned fusion features.
+
+Heavy deps (torchvision, supervision) are imported lazily inside the methods so importing
+the segmenter registry never pulls torch on a machine that lacks it (and tests can mock
+the seams). Licensing: torchvision detectors are BSD + COCO weights; ByteTrack is MIT
+(ADR-005; ultralytics' AGPL YOLO was dropped).
+"""
+import logging
+from typing import Any, Dict, List, Optional, Tuple
+
+import cv2
+import torch
+
+from backend.utils.geometry import get_velocity_normalised
+
+logger = logging.getLogger(__name__)
+
+# torchvision detection models are trained on COCO where the "person" category is
```

31. assistant (2026-06-13T19:18:16.381Z)

This is a very revealing find. The git history shows person_detector.py's module docstring (lines 18-21) explicitly claims: "Heavy deps (torchvision, supervision) are imported lazily inside the methods so importing the segmenter registry never pulls torch on a machine that lacks it." Yet line 27 does `import torch` at module top-level ? eagerly. So the docstring's own claim is contradicted by line 27, and this is pre-existing (introduced in PR #115, fusion P1, not this fusion-P2 PR).

The diff for this PR only ADDS the `detections_per_frame` method to person_detector.py ? it does NOT touch the `import torch` line. So the finding's claim that this is pre-existing and "neither adds nor removes torch from the registry path" is accurate.

Let me confirm the fusion_hybrid docstring claims are exactly as quoted, and the diff for `__init__.py`.

32. user (2026-06-13T19:18:18.561Z)

```
diff --git a/backend/pipeline/segmenters/__init__.py b/backend/pipeline/segmenters/__init__.py
index c003d1a..fc5de77 100644
--- a/backend/pipeline/segmenters/__init__.py
+++ b/backend/pipeline/segmenters/__init__.py
@@ -4,6 +4,7 @@ from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
 from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
 from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
 from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
+from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
```

```
 __all__ = [
     "segmenter_registry",
@@ -13,4 +14,5 @@ __all__ = [
     "HybridYoloSegmenter",
     "TrackNetHybridSegmenter",
     "WasbHybridSegmenter",
+    "FusionSegmenter",
 ]
=== status of fusion_hybrid (new file?) ===
?? backend/pipeline/detectors/fusion_features.py
?? backend/pipeline/segmenters/fusion_hybrid.py
```

33. assistant (2026-06-13T19:18:31.824Z)

All claims now fully verified. Let me assess the finding's accuracy against the code:

Empirically confirmed:
1. `import backend.pipeline.segmenters` ? torch in sys.modules: **True** (confirmed)
2. `import backend.pipeline.segmenters.fusion_hybrid` ? torch in sys.modules: **True** (confirmed, because importing any submodule runs the package `__init__.py` which imports yolo_hybrid)
3. Causal chain: `__init__.py:4` imports yolo_hybrid ? yolo_hybrid.py:25 top-level `from ...person_detector import PersonDetector` ? person_detector.py:27 `import torch` (eager). **Confirmed** by tracing.
4. fusion_hybrid's four named torch-free imports (wasb_runner, tracknet_runner, rally_gate, fusion_features) each show torch=False in isolation. **Confirmed.**
5. fusion_hybrid's PersonDetector import is correctly lazy inside `_person_features` (line 92, inside method). **Confirmed.**
6. The docstring claims at lines 12-13 ("the default path never imports torch / touches the GPU") and line 91 ("the default (person-OFF) path never pulls torch/cv2 through this module"). **Confirmed verbatim.**
7. Pre-existing condition: `import torch` at person_detector.py:27 predates this PR (introduced PR #115); this PR only adds a method. **Confirmed via git.**

The finding's nuance is precise and defensible. It correctly scopes the claim: fusion_hybrid's own module-level imports don't pull torch directly (the PersonDetector import IS lazy at line 92), so at the "this module" level the docstring is arguably accurate. The misleading framing is at the *registry/default-path* level ? importing the registry already loads torch via yolo_hybrid regardless of fusion. The finding self-correctly flags this as pre-existing and low severity.

Now, the adversarial test ? is this a real issue or a nitpick/false-positive? The finding is documentation accuracy only (no behavioral bug). It is correctly self-labeled "low" and even notes "No change required in fusion_hybrid." The line-91 docstring is narrowly defensible ("through this module"), but lines 12-13 ("the default path never imports torch / touches the GPU") overstate at the registry level. This is a real, verifiable documentation imprecision, but it is minor and the finding explicitly disclaims that the PR introduced any torch regression.

34. user (2026-06-13T19:18:48.723Z)

Structured output provided successfully

35. assistant (2026-06-13T19:18:56.040Z)

I've completed the adversarial verification.

The finding is **real (severity: low)**. I confirmed it both by reading the code and empirically (torch is actually installed in this environment, so the sys.modules test is meaningful):

- `import backend.pipeline.segmenters` and `import backend.pipeline.segmenters.fusion_hybrid` both leave torch in `sys.modules`.
- Cause traced exactly: `segmenters/__init__.py:4` ? `yolo_hybrid.py:25` top-level `from ...person_detector import PersonDetector` ? `person_detector.py:27` eager `import torch`.
- fusion_hybrid's own logic is clean: its four named imports are torch-free in isolation, and

its `PersonDetector` import is correctly lazy inside `_person_features` (line 92).
- The pre-existing attribution checks out via `git log -S` (line 27 came from PR #115; this PR only adds a method).

The finding is correctly scoped and self-labeled low. It's a documentation imprecision, not a regression. I also noted a reinforcing detail it under-states: person_detector.py's *own* docstring (lines 18-21) makes the same false "imported lazily / never pulls torch" claim while line 27 eagerly imports torch ? so the imprecision is systemic, and the real fix (if the guarantee is meant to hold) is to make yolo_hybrid + person_detector torch-imports lazy, both out of scope for this fusion-P2 PR.